This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Efficiently Reconstructing Dynamic Scenes One D4RT at a Time

Chuhan Zhang* Guillaume Le Moing* Skanda Koppula*◇ Ignacio Rocco*
Liliane Momeni* Junyu Xie°[1] Shuyang Sun* Rahul Sukthankar* Joëlle K. Barral*
Raia Hadsell* Zoubin Ghahramani* Andrew Zisserman*° Junlin Zhang*
Mehdi S. M. Sajjadi*[2]

*Google DeepMind ◇University College London °University of Oxford

Abstract

Understanding and reconstructing the complex geometry and motion of dynamic scenes from video remains a formidable challenge in computer vision. This paper introduces D4RT*, a simple yet powerful feedforward model designed to efficiently solve this task.* D4RT *utilizes a unified transformer architecture to jointly infer depth, spatio-temporal correspondence, and full camera parameters from a single video. Its core innovation is a novel querying mechanism that sidesteps the heavy computation of dense, per-frame decoding and the complexity of managing multiple, task-specific decoders. Our decoding interface allows the model to flexibly and independently probe the 3D position of any point in space and time. The result is a lightweight and highly scalable method that enables remarkably efficient training and inference. We demonstrate that our approach sets a new state of the art, outperforming previous methods across a wide spectrum of 4D reconstruction tasks. We refer to the project webpage for animated results.*[3]

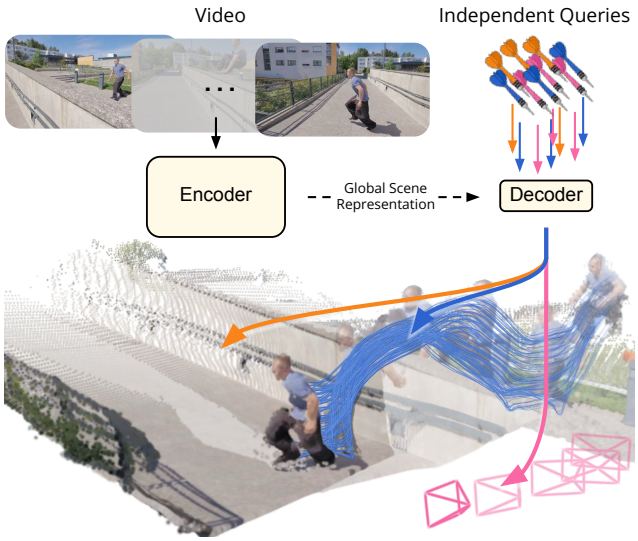


Figure 1. **D4RT** is a unified, efficient, feedforward method for ***D****ynamic* ***4D*** ***R****econstruction and* ***T****racking*, unlocking a variety of outputs including point cloud (), point tracks (), camera parameters () through a single interface.

1. Introduction

Traditional 3D reconstruction asks: *'What is the geometry of everything, everywhere, all at once?'* We argue this exhaustive, rigid approach is fundamentally ill-equipped for a dynamic world. Despite the clear need for unified 4D understanding, leading approaches often tackle the problem by dividing it into discrete, task-specific components.

For instance, MegaSaM [32] relies on a complex mosaic of off-the-shelf models to separately estimate mono-depth, metric depth, and motion segmentation. Fusing these disparate signals requires computationally expensive test-time optimization to enforce geometric consistency. Recent feedforward approaches such as VGGT [51] employ separate, specialized decoders for distinct modalities. Crucially, neither of these methods is capable of establishing correspondences for *dynamic* portions of the scene. While SpatialTrackerV2 [59] incorporates dynamics, it still lacks a unified, single-stage formulation, instead relying on costly iterative refinement.

We propose shifting the paradigm from fragmented, frame-level decoding to efficient, on-demand querying. We introduce D4RT, a feedforward method leveraging a flexible and scalable architecture to achieve full 4D reconstruction. As shown in Fig. 1, our model first encodes the input video, generating a latent scene representation which is then used to independently decode any number of spatio-temporal point queries. This simple, novel design unifies all 4D reconstruction tasks into a single interface and unlocks efficient training and inference.

[1]Work was done during an internship at Google DeepMind.
[2]Correspondence: d4rt@msajjadi.com
[3]Project website: https://d4rt-paper.github.io/

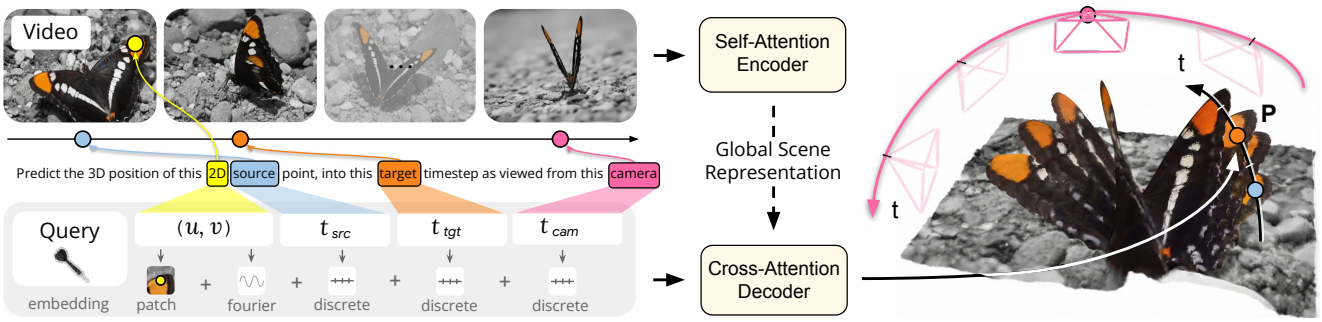


Figure 2. **D4RT model overview** – A global self-attention encoder first transforms the input video into the latent *Global Scene Representation* F, which is passed to a lightweight decoder. The decoder can be independently queried for the *3D position* $\mathbf{P}$ of any given 2D point (u, v) from the source timestep t_{src} at target timestep t_{tgt} in camera coordinate t_{cam}, unlocking full decoding at any point in space and time. The query also contains an embedding of the local video patch centered around (u, v), providing additional spatial context.

Our key contributions are as follows:

- We propose D4RT, a novel method for efficient feedforward querying of point-level 4D scene information captured in a video.
- We demonstrate how our unified approach unlocks 4D correspondence, point clouds, depth maps, and camera parameters for both static and dynamic scenes through a single interface.
- In an extensive set of experiments, we show that D4RT sets a new state of the art in *Dynamic 4D Reconstruction and Tracking* while outperforming existing approaches in both speed and accuracy.
- Finally, we demonstrate how our flexible decoder unlocks an efficient algorithm to track all pixels in a video, enabling dense, holistic scene reconstruction.

2. Method

D4RT is based on a simple encoder-decoder architecture inspired by the Scene Representation Transformer [42, 43]. As shown in Fig. 2, the video is first processed by a powerful encoder producing the *Global Scene Representation* F. The role of the encoder is to capture information about the full environment, identifying dense correspondence across all video frames, as well as understanding the flow of time and its effect on the scene. In a second stage, a lightweight decoder queries F through a unified interface. Specifically, given a 2D point in a *source frame*, the decoder predicts the *3D position* of the point at a given *target timestep* (defining the temporal state) and expressed relative to a given *camera* reference (defined by the frame timestep where the camera viewpoint corresponds to this reference).

We highlight three key properties of this design: first, indices need not coincide, fully disentangling space and time; second, independent query decoding enables efficient training, inference, and flexible (sparse or dense) decoding; and third, it unlocks diverse downstream applications in a unified manner (Tab. 1).

2.1. D4RT Framework

Given a video $V \in \mathbb{R}^{T \times H \times W \times 3}$, the encoder $\mathcal{E}$ extracts the latent *Global Scene Representation*

$$F = \mathcal{E}(V) \in \mathbb{R}^{N \times C}.$$

Once F is calculated, it remains fixed throughout the second stage, where the decoder $\mathcal{D}$ cross-attends from any number of queries into F. We define a query

$$\mathbf{q} = (u, v, t_{\text{src}}, t_{\text{tgt}}, t_{\text{cam}}),$$

where (u, v, t_{src}) correspond to *source* parameters and $(t_{\text{tgt}}, t_{\text{cam}})$ correspond to *target* parameters. Here, $(u, v) \in [0, 1]^2$ represent the normalized 2D coordinates of a point of interest in the source frame t_{src}, while $(t_{\text{tgt}}, t_{\text{cam}}) \in [1, \ldots, T]$ denote the temporal indices of the target timestep and the reference camera coordinate system (illustrated in Fig. 2). Each query $\mathbf{q}$ is processed *fully independently* with the video features F to produce its corresponding 3D point position $\mathbf{P}$:

$$\mathbf{P} = \mathcal{D}(\mathbf{q}, F) \in \mathbb{R}^3.$$

From query to 4D reconstruction. Through a simple variation of queries, our framework allows us to address a broad range of 4D tasks as shown in Tab. 1. Choosing any fixed point (u, v) from a source frame t_{src} in the video while varying $t_{\text{tgt}} = t_{\text{cam}} = \{1 \ldots T\}$ produces its *point track*, the 3D trajectory of the corresponding point throughout the video. For full *point cloud* reconstruction, the model directly predicts the 3D position of all pixels in the video in a shared reference frame t_{cam}. This alleviates the need for coordinate transformations to map pixels from different video frames into a unified coordinate system using explicit, potentially noisy camera estimates. *Depth maps* can be recovered by querying any pixel in the video with $t_{\text{src}} = t_{\text{tgt}} = t_{\text{cam}}$ and only keeping the Z-dimension of the output $\mathbf{P}$.

Task	Query				
	u	v	t_{src}	t_{tgt}	t_{cam}
Point Track	Fixed	Fixed	Fixed	$1 \dots T$	
Point Cloud	$1 \dots W$	$1 \dots H$	$1 \dots T$		Fixed
Depth Map	$1 \dots W$	$1 \dots H$	$1 \dots T$		
Extrinsics	$1 \dots h$	$1 \dots w$	Fixed		$1 \dots T$
Intrinsics	$1 \dots h$	$1 \dots w$	$1 \dots T$		

Table 1. **Unified decoding** – A diverse set of geometry-related tasks can be inferred by querying the Cartesian product of the respective entries. Note that for intrinsics and extrinsics, we only query a coarse (h, w) grid for faster inference.

We next detail how *camera extrinsics* and *intrinsics* predictions are obtained. To derive the relative camera pose between any pair of video frames $i, j \in [1 \dots T]$, we create queries for an ensemble of k source points $\{(u, v)\}$ sampled on a (h, w) grid in both reference frames:

$$\mathbf{q}_i = (u, v, i, i, i), \quad \mathbf{q}_j = (u, v, i, i, j).$$

The resulting sets $\{\mathcal{D}(\mathbf{q}_i, F)\}$ and $\{\mathcal{D}(\mathbf{q}_j, F)\}$ describe the same 3D points in different reference frames. We therefore only need to find the rigid transformation between them, which can be efficiently derived through Umeyama's algorithm [49] that solves a 3×3 SVD decomposition.

To recover intrinsics for video frame $i \in [1 \dots T]$, we construct a set of queries for different k source points $\{(u, v)\}$, again sampled on a (h, w) grid. We decode all corresponding 3D positions $\mathbf{P} = (p_x, p_y, p_z)$. Assuming a pinhole camera model with a principal point at $(0.5, 0.5)$, we get focal length parameters as follows:

$$f_x{=}p_z(u - 0.5)/p_x, \quad f_y{=}p_z(v - 0.5)/p_y.$$

We take the median over the k estimates for robustness. Camera models with distortion (*e.g.*, fisheye) can also be seamlessly incorporated by adding a non-linear refinement step on top of the initial estimation [64].

2.2. Model Architecture

Our encoder $\mathcal{E}$ is based on the Vision Transformer [11] with interleaved *local frame-wise*, and *global* self-attention layers [51]. For simplicity, and to support arbitrary aspect ratios, we resize the input video to a fixed square resolution before tokenizing it. To incorporate the original aspect ratio, we embed it into a separate token and pass it to the transformer along with the main video tokens.

Pointwise decoder. The decoder $\mathcal{D}$ is a small cross-attention transformer. A query token is first constructed by adding the Fourier feature embedding [50] of the 2D coordinates (u, v) to the learned discrete timestep embeddings for t_{src}, t_{tgt}, and t_{cam}. We empirically observe that augmenting the query with an embedding of the local 9×9 pixel RGB patch centered at (u, v) dramatically improves performance, see Sec. 4.4.

Algorithm 1 Efficient Dense Tracking of All Pixels.

Require: Input video V, model encoder $\mathcal{E}$ and decoder $\mathcal{D}$.
$F \leftarrow \mathcal{E}(V)$ ▷ Compute *Global Scene Representation*
$G \leftarrow \{\textbf{false}\}^{T\times H\times W}$ ▷ Initialize *Occupancy Grid*
$\mathcal{T} \leftarrow \emptyset$ ▷ Initialize *Set of Dense Tracks*
while any($G =$ **false**) **do**
  Sample a batch B of unvisited source points from G
  for each $(u, v, t_{\text{src}}) \in B$ **do** ▷ Process batch in parallel
    $Q \leftarrow \{u, v, t_{\text{src}}, t_{\text{tgt}}{=}t_{\text{cam}}{=}k\}_{k=1}^{T}$ ▷ Get *Track Queries*
    $P \leftarrow \{\mathcal{D}(\mathbf{q}_k, F)\}_{k=1}^{T}$ ▷ Run the decoder
    $G \leftarrow \text{Visible}(P)$ ▷ Set visible track pixels as visited
    $\mathcal{T} \leftarrow \mathcal{T} \cup P$ ▷ Add new track to the output
  end for
end while
return $\mathcal{T}$

Each query is separately decoded via cross-attention with the *Global Scene Representation* F, yielding an output feature that is then mapped to a 3D point position $\mathbf{P}$ through a simple learned projection. Processing queries independently through a lightweight decoder is a deliberate design decision with major advantages for training, inference, and general flexibility. During training, only a small, curated selection of all possible points needs to be decoded per video for supervision, minimizing both memory requirements and computational overhead. This independence also enables highly efficient inference thanks to trivial parallelization. Furthermore, queries can be chosen freely across all video frames, enabling new inference algorithms (Sec. 2.3) and making D4RT significantly faster than existing methods across several tasks (Fig. 3 and Tab. 3).

Interactions between queries would not only pose a computational burden, but also introduce train-test distribution shifts, as the model would overfit to the query distribution during training. Indeed, we empirically observed major inference performance drops when enabling self-attention between queries in early experiments. Our model achieves spatially and temporally consistent results not through heavy, coupled decoding mechanisms (*e.g.*, DPT [40], CoTracker [25]), but through the simple design choice of utilizing a lightweight decoder. This encourages the *encoder* to reconstruct the scene and implicitly encode the information in the global features F, which serves as an implicit means to enforce global consistency.

2.3. Training and Inference

The model is implemented in Kauldron [18] and trained end-to-end by minimizing the weighted sum of losses computed over a batch of N sampled queries. The primary supervision signal is derived from an L_1 loss applied to the

Model	Task		Functionality		Architecture	
	3D Reconstruction	Dynamic Correspondence	Flexible Ref. Frames	Sparse Decoding	Global Context	Single Decoder
MegaSaM [32]	✓	✗	✗	✗	✗	✗
DUSt3R[54]	✓	✗	✗	✗	✗	✗
VGGT [51]	✓	✗	✗	✗	✓	✗
π^3 [57]	✓	✗	✓	✗	✓	✗
St4RTrack [13]	✓	✓	✗	✗	✗	✗
STv2 [59]	✓	✓	✗	✓	✓	✗
D4RT (Ours)	✓	✓	✓	✓	✓	✓

Table 2. **Model capabilities** – We highlight both the tasks our model executes but also its comprehensive functionality and simple model architecture.

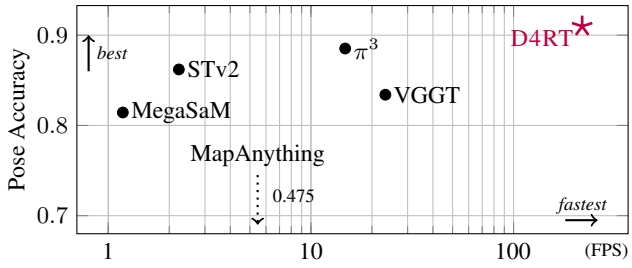


Figure 3. **Pose accuracy *vs*. speed** – D4RT achieves over 200 FPS for pose estimation on an A100 GPU, delivering superior accuracy while running 9× faster than VGGT and 100× faster than MegaSaM. Accuracy is defined as *1 – error*, averaged over ATE / RTE / RPE on Sintel and ScanNet.

normalized 3D point position $\mathbf{P}$. Specifically, both the target and the estimated point sets are normalized by their respective mean depths [54] and then passed through the transform $\text{sign}(x) \cdot \log(1+|x|)$ to dampen the influence of far-away points on the loss. We also supervise a set of auxiliary predictions from additional linear projections on the decoder output: An L_1 loss on *2D coordinates* of the point positions in image space; cosine similarity for *3D surface normals* [57]; binary cross-entropy for *target point visibility*; and L_1 on the vector of *point motion*. All loss terms are applied only where ground truth supervision is available. We finally incorporate a confidence penalty $-\log(c)$ where c additionally weights the 3D point error [28].

Efficient dense dynamic correspondence. A key capability of our query-based model is that it can efficiently compute dense correspondences for *all* pixels in a video, both static and dynamic. As shown in Fig. 4, this capability is crucial for building a complete, holistic scene reconstruction, which in turn is key to eliminating the occlusion-induced discontinuities and sparse artifacts common in prior works. However, a naive approach to reconstruct tracks for all pixels across the video would involve $O(T^2HW)$ queries, the majority of which are not required.

We introduce Alg. 1 which exploits spatio-temporal redundancy using an occupancy grid $G \in \{0,1\}^{T\times H\times W}$ to speed this procedure up significantly. The algorithm only initiates new tracks from unvisited pixels. Each *full-video track* marks all spatio-temporal pixels it *visibly* passes through as visited. Empirically, we find that this yields an adaptive speedup between 5–15× depending on the motion complexity in the video. This dense, flexible strategy is feasible for our method precisely because our decoder is *both* sparse, and lightweight. It is notable that prior works are ill-suited for this task for various reasons. Most methods fail to provide any correspondences for dynamic portions of the scenes [32, 51, 57], dense frame-level decoding remains locked in the costly naive approach [13], and models with heavy sparse decoders face a large per-query cost [26, 59].

3. Related Work

Classical approaches to 3D reconstruction are fundamentally anchored in multi-view geometry, specifically Structure-from-Motion (SfM) [21, 36] and Multi-View Stereo (MVS) [14, 46]. The standard pipeline exemplified by COLMAP [45] incrementally estimates and optimizes sparse geometry and camera poses. The advantage of these methods is their explicit enforcement of geometric consistency and mathematically interpretable reconstructions. However, they are computationally intensive and brittle.

3.1. Feedforward 3D Reconstruction

The field of 3D reconstruction has recently shifted towards end-to-end, feedforward models that directly infer geometry from images. The seminal work DUSt3R [54] demonstrated that Transformer-based networks [50] can perform 3D reconstruction from unposed and uncalibrated image pairs in an end-to-end approach. VGGT [51] later scaled this approach beyond pairs by using a Vision Transformer [11] with global attention. Building on this feedforward paradigm, several works have extended the methodology to dynamic videos [5, 53, 55, 63] and more efficient inference [57, 60]. However, these models share significant limitations. For instance, many do not support changing camera intrinsics within a video [32, 51, 57], and several are restricted to using only the first frame as the camera reference [32, 53, 54]. More importantly, architectures inspired by VGGT [51] either use separate decoder heads for each task [53, 57, 63] (*e.g.*, depth, pose, point cloud), or they incorporate separate models for sub-tasks of 4D reconstruction [34, 47, 55], making the full pipeline cumbersome and computationally expensive to run. Finally, these methods are not directly capable of providing correspondences for dynamic regions of the scene.

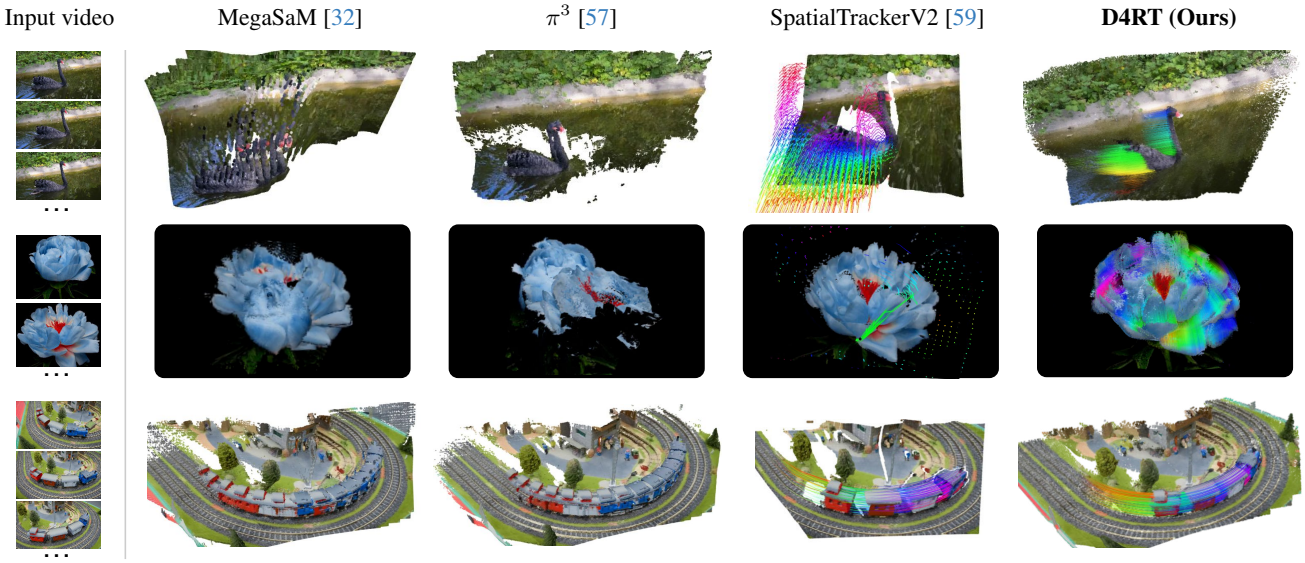


Figure 4. **Reconstruction results across methods** – Pure reconstruction methods (MegaSaM and π^3) are only able to accumulate point clouds of all pixels; exhibiting clear failure cases in dynamic scenes. For example, the swan is repeated in MegaSaM's reconstruction, and π^3 is failing entirely to reconstruct the flower. SpatialTrackerV2, a state-of-the-art tracking method, successfully captures dynamics, however its design only allows tracking points from *one* frame, leaving gaps in the reconstruction (behind the swan and train). D4RT is the only method that successfully reconstructs a full 4D representation of the scene including *all* pixels of the video.

3.2. From 2D to 3D Point Tracking

The task of point tracking aims to establish long-term 2D correspondences through challenges like occlusions and non-rigid motion, evolving from early methods such as Particle Video [44] to newer deep-learning approaches [1, 2, 9, 10, 19, 66]. This field has further progressed from tracking a sparse set of points to the dense tracking of every pixel in a video [20, 31, 35]. A parallel line of work has explored lifting these 2D tracks into 3D, which is achieved by projecting tracks from image space to camera coordinates using ground truth or predicted depth and camera intrinsics [30].

Most recently, models like SpatialTracker [58, 59], L4P [3], DPM [47] and St4RTracker [13] have moved to directly predicting 3D tracks without priors. Yet, limitations remain: St4RTracker [13] and DPM [47] follow the pairwise paradigm of DUSt3R [54], preventing holistic video processing. SpatialTrackerV2 [59] is a multi-stage approach and exhibits slow inference speeds as it relies on iterative refinements. Meanwhile, L4P [3] requires different heads for sparse and dense outputs of the same nature (*e.g.*, flow vs. 2D tracking or depth vs. 3D tracking). In contrast, our approach unifies these tasks within a single architecture. Tab. 2 summarizes capabilities across a recent set of state-of-the-art models.

4. Experiments

We benchmark D4RT against recent state-of-the-art methods. After inspecting qualitative differences in capability in Sec. 4.1, we proceed with evaluating 4D Reconstruction and Tracking performance in Sec. 4.2, focusing the comparison on the best methods from Sec. 3.2. We then continue with pure reconstruction tasks in Sec. 4.3, comparing to the recent SOTA models mentioned in Sec. 3.1. We conclude with a set of key ablations in Sec. 4.4.

Method	Max. Track Count @ Target FPS			
	60 FPS	24 FPS	10 FPS	1 FPS
DELTA [35]	0	5	408	5,770
SpatialTrackerV2 [59]	29	84	219	2,290
D4RT (Ours)	550	1,570	3,890	40,180

Table 3. **3D tracking throughput** – We measure the maximum number of full-video 3D point tracks that different model can produce while maintaining a given FPS target on a single A100 GPU. Note that for D4RT, each track consists of T independent queries processed by the decoder. D4RT is 18–300× faster than others.

Training setup. For the encoder, we use the ViT-g model variant with 40 layers on a spatio-temporal patch size of $2\times16\times16$. This encoder and our 8-layer cross-attention decoder contain 1 B and 144 M parameters, respectively. Our training mixture consists of the publicly available BlendedMVS [61], Co3Dv2 [41], Dynamic Replica [24], Kubric [17], MVS-Synth [22], PointOdyssey [65], ScanNet++ [62], ScanNet [6], Tartanair [56], VirtualKitti [15], Waymo Open [48], as well as internal datasets. We train on 48-frame clips at 256×256 resolution, decoding 2048



Figure 5. **Visualizations on in-the-wild videos** – D4RT demonstrates accurate reconstructions on static (top row) and dynamic scenes (bottom row). In the presence of motion, D4RT additionally produces robust 3D point trajectories.

w/ GT intrin.	Method	TAPVid-3D												
		Camera Coordinate 3D tracking									World Coordinate 3D tracking			
		DriveTrack			ADT			PStudio			DriveTrack		ADT	
		AJ ↑	APD_{3D} ↑	OA ↑	AJ ↑	APD_{3D} ↑	OA ↑	AJ ↑	APD_{3D} ↑	OA ↑	APD_{3D} ↑	L1 ↓	APD_{3D} ↑	L1 ↓
✗	St4RTrack [13]	-	-	-	-	-	-	-	-	-	0.020	0.499	0.062	0.839
	CoTracker3 [26] + UniDepthV2 [39]	0.038	0.062	0.856	0.102	0.146	0.937	0.119	0.176	0.862	-	-	-	-
	CoTracker3 [26] + VGGT [51]	0.129	0.189	0.856	0.132	0.190	0.937	0.045	0.070	0.862	0.205	0.170	0.192	0.736
	SpatialTrackerV2 [59]	0.064	0.100	0.865	0.260	0.342	0.936	0.097	0.152	0.805	0.101	0.139	0.336	0.238
	D4RT (Ours)	0.257	0.345	0.875	0.240	0.319	0.926	0.138	0.186	0.897	0.373	0.020	0.319	0.096
✓	CoTracker3 [26] + UniDepthV2 [39]	0.147	0.225	0.856	0.173	0.254	0.937	0.205	0.311	0.862	-	-	-	-
	CoTracker3 [26] + VGGT [51]	0.245	0.342	0.856	0.175	0.250	0.937	0.215	0.318	0.862	0.292	0.160	0.234	0.755
	DELTA [35]	0.170	0.238	0.874	0.199	0.258	0.879	0.175	0.261	0.760	-	-	-	-
	SpatialTrackerV2 [59]	0.195	0.275	0.865	0.303	0.404	0.936	0.175	0.270	0.805	0.201	0.117	0.378	0.263
	D4RT (Ours)	0.304	0.410	0.875	0.307	0.408	0.926	0.372	0.498	0.897	0.470	0.017	0.398	0.093

Table 4. **4D reconstruction and tracking** – We evaluate 3D tracking capability on dynamic videos, with tracks predicted in both local camera coordinates (left) and world coordinates (right). Our model achieves superior performance compared to the prior state-of-the-art.

random queries that are oversampled on specific regions for more efficient training, see the appendix for details. The model is trained for 500 k steps using the AdamW optimizer [29, 33] with a local batch size of 1 across 64 TPU chips, taking a total of just over 2 days to complete.

4.1. Qualitative Analysis

We begin our evaluations by inspecting video reconstruction quality in Fig. 4. Dynamic objects (*e.g.*, swan and train) reveal qualitative differences, highlighting performance gaps between different methods. Pure reconstruction methods (see Sec. 3.1), exemplified by the state-of-the-art models MegaSaM and π^3, fail to understand dynamics in the scenes, producing either repeated images of the entities as they move through the scene, or failing to reconstruct them altogether. Conversely, tracking-based models successfully reconstruct and track dynamics. However, they commonly only track points from one frame, leaving gaps in the reconstruction in all areas that are occluded in the first frame as seen for SpatialTrackerV2 [59] here. Meanwhile, D4RT is able to track all dynamic pixels of the video in a unified reference frame. We further evaluate our method on diverse in-the-wild videos in Fig. 5, demonstrating its performance to both static and dynamic scenes.

4.2. 4D Reconstruction and Tracking

We now turn to the quantitative evaluation of 4D Reconstruction and Tracking, comparing D4RT to other methods that can estimate dynamic correspondences. We evaluate on TAPVid-3D [30] which is comprised of three subsets of real-world videos in challenging settings.

We first evaluate local-frame 3D tracking using the standard protocol from Koppula et al. [30], measuring the ability to track every observed point's 3D position in the *local* camera reference frames. We report standard 3D tracking metrics *Average percent of points within delta error* APD_{3D}, *Occlusion Accuracy* (OA) and *3D Average Jaccard* (AJ). As shown in Tab. 4 (left), D4RT achieves state-

Method	Point Cloud		Video Depth							
	Sintel	ScanNet	Sintel		ScanNet		KITTI		Bonn	
	L1 ↓	L1 ↓	AbsRel (S) ↓	AbsRel (SS) ↓	AbsRel (S) ↓	AbsRel (SS) ↓	AbsRel (S) ↓	AbsRel (SS) ↓	AbsRel (S) ↓	AbsRel (SS) ↓
MegaSaM [32]	1.531	0.072	0.342	0.249	0.050	0.047	0.109	0.101	0.056	0.056
VGGT [51]	1.582	0.063	0.318	0.247	0.044	0.040	0.094	0.067	0.055	0.051
MapAnything [27]	1.718	0.064	0.397	0.306	0.043	0.035	0.096	0.090	0.076	0.049
SpatialTrackerV2 [59]	1.375	0.036	0.209	0.175	0.027	0.025	0.075	0.064	0.042	0.978
π^3 [57]	1.139	0.030	0.241	0.163	0.021	0.019	0.055	0.053	0.033	0.032
D4RT (Ours)	0.768	0.028	0.171	0.148	0.020	0.018	0.055	0.051	0.036	0.036

Table 5. **Video depth and point map estimation** – Quantitative results for both video depth and point map estimation across four benchmarks. D4RT achieves top-tier performance on the depth estimation task under both scale-only (S) and scale-and-shift (SS) alignments.

Method	Sintel			ScanNet			Re10K
	ATE ↓	RPE-T ↓	RPE-R ↓	ATE ↓	RPE-T ↓	RPE-R ↓	Pose AUC ↑
MegaSaM [32]	0.074	0.030	0.126	0.029	0.016	0.839	71.0
VGGT [51]	0.168	0.056	0.428	0.016	0.012	0.316	70.2
MapAnything [27]	0.202	0.089	2.383	0.023	0.016	0.438	68.7
SpatialTrackerV2 [59]	0.126	0.053	1.052	0.018	0.012	0.324	75.7
π^3 [57]	0.086	0.039	0.248	0.015	0.010	0.291	78.7
D4RT (Ours)	0.065	0.024	0.126	0.014	0.010	0.302	83.5

Table 6. **Camera pose estimation** – We evaluate D4RT against state-of-the-art methods on static indoor scenes (ScanNet, Re10K) and dynamic outdoor scenes (Sintel).

of-the-art results against competing methods, both with and without known ground-truth intrinsics.

Tab. 4 (right) shows results for *world coordinate 3D tracking*. This task measures a model's ability to predict tracks within a single, consistent world coordinate system, thereby also measuring the models' ability to implicitly change reference frames. PStudio is omitted here as it has no camera motion, rendering this task effectively identical to the preceding one. Since there is no concept of occlusion for this task, we report APD_{3D} as well as the L1 distance between the predicted and ground-truth tracks which provides a comprehensive, global measure of deviation. We observe that our model excels at this task as well, exhibiting strong improvements across metrics.

We finally compare efficiency in Tab. 3, by counting the number of tracks each model can produce for a given target frame-per-second rate (FPS). The measurements show that our model is 18-300× faster than prior methods.

4.3. 3D Reconstruction

We now evaluate the 3D reconstruction capabilities of our model. The most holistic task here is 3D point cloud reconstruction, which requires predicting all observed pixels in the same world coordinate system.

We evaluate on two standard benchmarks: MPI Sintel [4] and ScanNet [7], which consist of dynamic and static scenes, respectively. For evaluation, we first align the predicted and ground-truth point clouds via mean-shifting following Li et al. [32] before reporting the mean L1 distance. As shown in Tab. 5 (left), our model outperforms all recent state-of-the-art models across datasets.

Depth estimation. We evaluate depth estimation performance across four datasets: Sintel [4], ScanNet [7], KITTI [16], and Bonn [37]. Following prior works [32, 53], we align the predicted video depth with the ground-truth depth before computing metrics. Specifically, we adopt two alignment settings: *scale-only* (S) and *scale-and-shift* (SS) alignment. Both settings determine a single global scale factor between the predicted and ground-truth depths across the sequence, while the latter introduces an additional 3D translation term, following an affine-invariant setup. After alignment, we compute the *Absolute Relative Error* (AbsRel) [12] between the predicted and ground-truth depths.

We show results in Tab. 5 (right). D4RT achieves top-tier performance across all benchmarks, with particularly strong results on Sintel which is the most challenging dataset due to its highly dynamic scenes.

Camera pose estimation. We evaluate camera pose estimation performance on the same datasets. Following prior work [23, 53], we evaluate on the 14-sequence subset of Sintel which provides full rendering with motion blur and atmospheric effects to measure in-the-wild performance. We report the *Absolute Translation Error* (ATE), *Relative Translation Error* (RPE trans), and *Relative Rotation Error* (RPE rot) after Sim(3) alignment with the ground truth as in [53, 57, 63], as well as the *Pose AUC*@30 as in [51]. Tab. 6 presents the results, where our model strongly outperforms competing baselines on both indoor scenes (ScanNet and Re10K) and simulated outdoor scenes (Sintel).

Fig. 3 visualizes efficiency and accuracy for camera pose estimation across several recent state-of-the-art methods. To ensure a fair comparison, we remove the decoding heads of the baselines which are unrelated to camera estimation, making them faster. Despite this, we find that D4RT surpasses all existing methods in both accuracy and efficiency, notably outperforming MegaSaM in throughput by two orders of magnitude.

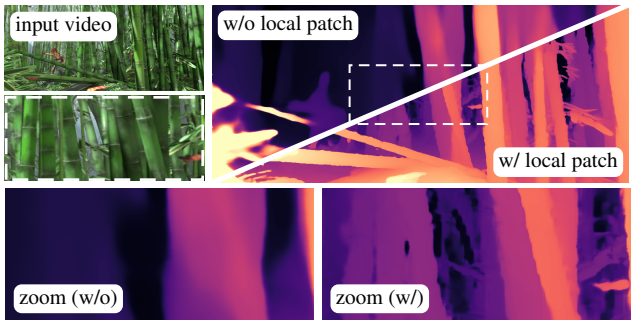


Figure 6. **Preservation of low-level details** – We ablate the effect of including local patch information into the model's queries, visualizing the resulting depth maps on an example from the Sintel dataset. We find that the local RGB patches help preserve fine-grained details and produce sharper object boundaries.

w/ local appear. patch	Video Depth		Camera Pose		
	AbsRel (S) ↓	AbsRel (SS) ↓	ATE ↓	RPE-T ↓	RPE-R ↓
✗	0.366	0.306	0.173	0.031	0.262
✓	0.302	0.257	0.091	0.028	0.245

Table 7. **Local RGB patch** – We evaluate a ViT-L model, trained with and without the appearance patch as input to the decoder, on Sintel video depth and camera pose estimation.

4.4. Ablation Studies

We present a series of ablation studies to validate our design choices and hyperparameters. Unless otherwise noted, we adopt a ViT-L backbone with 6 decoder layers as the default setting. The model is trained for 300 k iterations with a batch size 16. All results are reported on the challenging Sintel dataset for both video depth and camera pose.

Local RGB patch. As described in Sec. 2.2, we introduce an additional embedding that encodes the local appearance around the query's source location. This additional embedding yields dramatic performance improvements as shown in Tab. 7 and Fig. 6. The improvements can be attributed to two factors: (i) the local appearance information helps the query establish more reliable correspondences with the encoded spatiotemporal features; and (ii) these patches provide low-level cues that help to segment objects from their surroundings, leading to fine-grained predictions, as exemplified by the sharper depth estimates. We demonstrate how the patches unlock subpixel precision in Appendix Sec. C.

Encoder size. We now examine how performance scales with the size of our encoder. Tab. 8 summarizes our findings for ViT-B (90 M) to ViT-g (1 B). As the number of trainable parameters increases, we observe a significant performance improvement, particularly in depth estimation and RPE-R in camera pose estimation.

Pretrained encoder. We initialize our video encoder using publicly available pre-trained VideoMAEv2 [52] weights.

Backbone size	Video Depth		Camera Pose		
	AbsRel (S) ↓	AbsRel (SS) ↓	ATE ↓	RPE-T ↓	RPE-R ↓
ViT-B	0.319	0.232	0.145	0.034	0.266
ViT-L	0.256	0.214	0.073	0.027	0.191
ViT-H	0.226	0.173	0.070	0.028	0.186
ViT-g	0.191	0.168	0.078	0.026	0.160

Table 8. **Backbone size** – We examine how D4RT scales with the size of the pretrained ViT encoder backbone, evaluating video depth and camera pose estimation on Sintel. Performance clearly improves as the backbone increases from ViT-B to ViT-g.

Losses	Video Depth		Camera Pose		
	AbsRel(S) ↓	AbsRel(SS) ↓	ATE ↓	RPE-T ↓	RPE-R ↓
D4RT	0.302	0.257	0.091	0.028	0.245
w/o 2D position	+0.071	+0.063	+0.002	+0.002	-0.028
w/o normal	+0.043	+0.026	+0.003	+0.003	-0.022
w/o displacement	+0.011	+0.007	+0.011	+0.003	+0.007
w/o visibility	-0.003	-0.025	+0.012	+0.011	+0.022
w/o confidence	+0.002	-0.025	+0.126	+0.061	+0.115

Table 9. **Auxiliary losses** – We remove auxiliary losses individually to evaluate their impact on model performance. A slight trade-off between depth and camera estimation pose is observed, though we find that all auxiliary losses improve overall performance.

This initialization is critical to the model's success; replacing it with random weights leads to a significant performance drop, as detailed in Sec. D in the appendix.

Training data. To address reproducibility and isolate the impact of our internal data, we ablate a variant of D4RT trained exclusively on publicly available datasets. As detailed in Sec. D if the appendix, D4RT retains its state-of-the-art performance and continues to outperform competing baselines even in this restricted data regime.

Auxiliary losses. We finally investigate the contribution of each auxiliary loss in Tab. 9. We observe varied behaviors: some significantly boost depth performance (*e.g.*, 2D position, normal), others are critical for better camera pose estimation (*e.g.*, confidence), and displacement has small improvements across the board.

5. Conclusion

We introduce D4RT, a simple yet highly scalable feedforward model that reconstructs dynamic 4D scenes with temporal correspondence. Our core innovation is an efficient, query-based decoder that allows for the independent prediction of any point's 3D position across time. This flexible parametrization avoids the computational bottlenecks of dense per-frame decoders, enabling inference that scales linearly with the number of points to be reconstructed. We demonstrate that D4RT achieves state-of-the-art results across a wide range of 4D tasks including depth, point cloud estimation, and 3D point tracking. D4RT offers a unified framework for the next generation of 4D perception.

Contribution and Acknowledgements. MS led the project, with management support from JZ. MS proposed the SRT-style decoder, and GL proposed local RGB patches and tracking-all-pixels. CZ, GL, IR, and MS contributed to the core model design. LM, SS, IR, and SK designed and created training datasets. CZ, GL, and MS carried out the major implementation, with significant contributions from SK, IR, LM, and JX. CZ drove the model experimentation. Comprehensive evaluations and data pipelines were set up by CZ, GL, SK, IR, LM, JX, SS, and MS. Visualizations were produced by GL, IR and JX. RS, JB, RH, ZG, and AZ provided project support, advising, and guidance.

We thank a number of colleagues and advisors for making this work possible. We thank Saurabh Saxena, Kaiming He, Carl Doersch, Leonidas Guibas, Noah Snavely, Ben Poole, Joao Carreira, Pauline Luc, Yi Yang, Howard Huang, Huizhong Chen, Cordelia Schmid for providing advice during the project; Gabriel Brostow, for advising SK during the course of the project and for providing feedback on the manuscript; Relja Arandjelović and Maks Ovsjanikov for providing feedback on the manuscript; Ross Goroshin, Tengda Han and Dilara Gokay for their help during early-stage development; Aravindh Mahendran for helping with code reviews; Daniel Duckworth for helping with visualizations and comparisons against baselines; and Alberto García and Jesús Pérez for the invaluable contributions to data generation and collection. Finally, we thank the anonymous CVPR reviewers for their feedback on the manuscript.

References

[1] Görkay Aydemir, Xiongyi Cai, Weidi Xie, and Fatma Güney. Track-On: Transformer-based online point tracking with memory. In *ICLR*, 2025. 5

[2] Görkay Aydemir, Weidi Xie, and Fatma Güney. Track-on2: Enhancing online point tracking with memory. *arXiv preprint*, 2025. 5

[3] Abhishek Badki, Hang Su, Bowen Wen, and Orazio Gallo. L4P: Towards unified low-level 4D vision perception. In *3DV*, 2026. 5

[4] Daniel J. Butler, Jonas Wulff, Garrett B. Stanley, and Michael J. Black. A naturalistic open source movie for optical flow evaluation. In *ECCV*, 2012. 7

[5] Xingyu Chen, Yue Chen, Yuliang Xiu, Andreas Geiger, and Anpei Chen. Easi3r: Estimating disentangled motion from dust3r without training. *ICCV*, 2025. 4

[6] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *CVPR*, 2017. 5

[7] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *CVPR*, 2017. 7

[8] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. VGGT-Long: Chunk it, loop it, align it – pushing VGGT's limits on kilometer-scale long RGB sequences. *arXiv preprint*, 2025. 13

[9] Carl Doersch, Ankush Gupta, Larisa Markeeva, Adria Recasens, Lucas Smaira, Yusuf Aytar, Joao Carreira, Andrew Zisserman, and Yi Yang. Tap-vid: A benchmark for tracking any point in a video. *NeurIPS*, 2022. 5

[10] Carl Doersch, Yi Yang, Mel Vecerik, Dilara Gokay, Ankush Gupta, Yusuf Aytar, Joao Carreira, and Andrew Zisserman. Tapir: Tracking any point with per-frame initialization and temporal refinement. In *ICCV*, 2023. 5

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 3, 4

[12] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *NeurIPS*, 2014. 7

[13] Haiwen Feng, Junyi Zhang, Qianqian Wang, Yufei Ye, Pengcheng Yu, Michael J Black, Trevor Darrell, and Angjoo Kanazawa. St4rtrack: Simultaneous 4d reconstruction and tracking in the world. *ICCV*, 2025. 4, 5, 6

[14] Yasutaka Furukawa, Carlos Hernández, et al. Multi-view stereo: A tutorial. *Foundations and trends® in Computer Graphics and Vision*, 2015. 4

[15] A Gaidon, Q Wang, Y Cabon, and E Vig. Virtual worlds as proxy for multi-object tracking analysis. In *CVPR*, 2016. 5

[16] Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics: The KITTI dataset. *IJRR*, 2013. 7, 13

[17] Klaus Greff, Francois Belletti, Lucas Beyer, Carl Doersch, Yilun Du, Daniel Duckworth, David J Fleet, Dan Gnanapragasam, Florian Golemo, Charles Herrmann, et al. Kubric: A scalable dataset generator. In *CVPR*, 2022. 5

[18] Klaus Greff, Etienne Pot, and Mehdi S. M. Sajjadi. Kauldron: A neural network training framework, optimized for research velocity and modularity. https://github.com/google-research/kauldron, 2023. 3

[19] Adam W Harley, Zhaoyuan Fang, and Katerina Fragkiadaki. Particle video revisited: Tracking through occlusions using point trajectories. In *ECCV*, 2022. 5

[20] Adam W. Harley, Yang You, Xinglong Sun, Yang Zheng, Nikhil Raghuraman, Yunqi Gu, Sheldon Liang, Wen-Hsuan Chu, Achal Dave, Pavel Tokmakov, Suya You, Rares Ambrus, Katerina Fragkiadaki, and Leonidas J. Guibas. AllTracker: Efficient dense point tracking at high resolution. In *ICCV*, 2025. 5

[21] Richard Hartley and Andrew Zisserman. *Multiple view geometry in computer vision*. Cambridge university press, 2003. 4

[22] Po-Han Huang, Kevin Matzen, Johannes Kopf, Narendra Ahuja, and Jia-Bin Huang. Deepmvs: Learning multi-view stereopsis. In *CVPR*, 2018. 5

[23] Zeren Jiang, Chuanxia Zheng, Iro Laina, Diane Larlus, and Andrea Vedaldi. Geo4d: Leveraging video generators for geometric 4d scene reconstruction. In *ICCV*, 2025. 7

[24] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Dynamicstereo: Consistent dynamic depth from stereo videos. In *CVPR*, 2023. 5

[25] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Cotracker: It is better to track together. In *ECCV*, 2024. 3, 15

[26] Nikita Karaev, Yuri Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Cotracker3: Simpler and better point tracking by pseudo-labelling real videos. In *ICCV*, 2025. 4, 6

[27] Nikhil Keetha, Norman Müller, Johannes Schönberger, Lorenzo Porzi, Yuchen Zhang, Tobias Fischer, Arno Knapitsch, Duncan Zauss, Ethan Weber, Nelson Antunes, et al. Mapanything: Universal feed-forward metric 3d reconstruction. *arXiv preprint*, 2025. 7

[28] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? In *NeurIPS*, 2017. 4

[29] Diederik P Kingma. Adam: A method for stochastic optimization. *ICLR*, 2015. 6

[30] Skanda Koppula, Ignacio Rocco, Yi Yang, Joe Heyward, Joao Carreira, Andrew Zisserman, Gabriel Brostow, and Carl Doersch. Tapvid-3d: A benchmark for tracking any point in 3d. In *NeurIPS*, 2024. 5, 6

[31] Guillaume Le Moing, Jean Ponce, and Cordelia Schmid. Dense optical tracking: Connecting the dots. In *CVPR*, 2024. 5

[32] Zhengqi Li, Richard Tucker, Forrester Cole, Qianqian Wang, Linyi Jin, Vickie Ye, Angjoo Kanazawa, Aleksander Holynski, and Noah Snavely. MegaSaM: Accurate, fast and robust structure and motion from casual dynamic videos. In *CVPR*, 2025. 1, 4, 5, 7, 17

[33] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2019. 6

[34] Jiahao Lu, Tianyu Huang, Peng Li, Zhiyang Dou, Cheng Lin, Zhiming Cui, Zhen Dong, Sai-Kit Yeung, Wenping Wang, and Yuan Liu. Align3r: Aligned monocular depth estimation for dynamic videos. In *CVPR*, 2025. 4

[35] Tuan Duc Ngo, Peiye Zhuang, Chuang Gan, Evangelos Kalogerakis, Sergey Tulyakov, Hsin-Ying Lee, and Chaoyang Wang. Delta: Dense efficient long-range 3d tracking for any video. *ICLR*, 2025. 5, 6

[36] John Oliensis. A critique of structure-from-motion algorithms. *CVIU*, 2000. 4

[37] Emanuele Palazzolo, Jens Behley, Philipp Lottes, Philippe Giguère, and Cyrill Stachniss. Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals. In *IROS*, 2019. 7

[38] Duc-Hai Pham, Tung Do, Phong Nguyen, Binh-Son Hua, Khoi Nguyen, and Rang Nguyen. SharpDepth: Sharpening metric depth predictions using diffusion distillation. In *CVPR*, 2025. 14

[39] Luigi Piccinelli, Christos Sakaridis, Yung-Hsu Yang, Mattia Segu, Siyuan Li, Wim Abbeloos, and Luc Van Gool. UniDepthV2: Universal Monocular Metric Depth Estimation Made Simpler. *PAMI*, 2025. 6

[40] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. *ICCV*, 2021. 3

[41] Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction. In *ICCV*, 2021. 5

[42] Mehdi SM Sajjadi, Henning Meyer, Etienne Pot, Urs Bergmann, Klaus Greff, Noha Radwan, Suhani Vora, Mario Lučić, Daniel Duckworth, Alexey Dosovitskiy, et al. Scene representation transformer: Geometry-free novel view synthesis through set-latent scene representations. In *CVPR*, 2022. 2

[43] Mehdi S. M. Sajjadi, Aravindh Mahendran, Thomas Kipf, Etienne Pot, Daniel Duckworth, Mario Lučić, and Klaus Greff. RUST: Latent Neural Scene Representations from Unposed Imagery. *CVPR*, 2023. 2

[44] Peter Sand and Seth Teller. Particle video: Long-range motion estimation using point trajectories. *IJCV*, 2008. 5

[45] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *CVPR*, 2016. 4

[46] Johannes L Schönberger, Enliang Zheng, Jan-Michael Frahm, and Marc Pollefeys. Pixelwise view selection for unstructured multi-view stereo. In *ECCV*, 2016. 4

[47] Edgar Sucar, Zihang Lai, Eldar Insafutdinov, and Andrea Vedaldi. Dynamic point maps: A versatile representation for dynamic 3d reconstruction. *arXiv preprint*, 2025. 4, 5

[48] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, Vijay Vasudevan, Wei Han, Jiquan Ngiam, Hang Zhao, Aleksei Timofeev, Scott Ettinger, Maxim Krivokon, Amy Gao, Aditya Joshi, Yu Zhang, Jonathon Shlens, Zhifeng Chen, and Dragomir Anguelov. Scalability in perception for autonomous driving: Waymo open dataset. In *CVPR*, 2020. 5

[49] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *TPAMI*, 1991. 3, 13

[50] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 3, 4

[51] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *CVPR*, 2025. 1, 3, 4, 6, 7, 15

[52] Limin Wang, Bingkun Huang, Zhiyu Zhao, Zhan Tong, Yinan He, Yi Wang, Yali Wang, and Yu Qiao. VideoMAE v2: Scaling video masked autoencoders with dual masking. In *CVPR*, 2023. 8, 15

[53] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. In *CVPR*, 2025. 4, 7

[54] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *CVPR*, 2024. 4, 5

[55] Shizun Wang, Zhenxiang Jiang, Xingyi Yang, and Xinchao Wang. C4d: 4d made from 3d through dual correspondences. In *ICCV*, 2025. 4

[56] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. TartanAir: A dataset to push the limits of visual slam. In *IROS*, 2020. 5

[57] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. pi^3: Permutation-equivariant visual geometry learning. *arXiv preprint*, 2025. 4, 5, 7, 15, 17

[58] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. Spatialtracker: Tracking any 2d pixels in 3d space. In *CVPR*, 2024. 5

[59] Yuxi Xiao, Jianyuan Wang, Nan Xue, Nikita Karaev, Yuri Makarov, Bingyi Kang, Xing Zhu, Hujun Bao, Yujun Shen, and Xiaowei Zhou. SpatialTrackerV2: Advancing 3d point tracking with explicit camera motion. In *ICCV*, 2025. 1, 4, 5, 6, 7, 15, 17

[60] Jianing Yang, Alexander Sax, Kevin J Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *CVPR*, 2025. 4

[61] Yao Yao, Zixin Luo, Shiwei Li, Jingyang Zhang, Yufan Ren, Lei Zhou, Tian Fang, and Long Quan. Blendedmvs: A large-scale dataset for generalized multi-view stereo networks. In *CVPR*, 2020. 5

[62] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *ICCV*, 2023. 5

[63] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. Monst3r: A simple approach for estimating geometry in the presence of motion. *ICLR*, 2025. 4, 7

[64] Zhengyou Zhang. A flexible new technique for camera calibration. *TPAMI*, 2002. 3

[65] Yang Zheng, Adam W. Harley, Bokui Shen, Gordon Wetzstein, and Leonidas J. Guibas. Pointodyssey: A large-scale synthetic dataset for long-term point tracking. In *ICCV*, 2023. 5

[66] Artem Zholus, Carl Doersch, Yi Yang, Skanda Koppula, Viorica Patraucean, Xu Owen He, Ignacio Rocco, Mehdi SM Sajjadi, Sarath Chandar, and Ross Goroshin. TAPNext: Tracking any point as next token prediction. In *ICCV*, 2025. 5